

Home-Lab SIEM Detection Build with Wazuh

Setup, custom detection rules and triggered-alert evidence

Prepared by:	Mahnoor Mehtab Abbasi
Programme:	SafeX Cybersecurity Internship - Week 2 Task
Date:	August 2026

Table of Contents

1. What this report covers.....	3
2. Lab architecture.....	3
3. Bringing the Wazuh server up.....	5
4. Network path between the two machines.....	8
5. Deploying and enrolling the agent.....	9
6. Generating baseline events before writing any rules.....	12
7. Writing the detection rules.....	15
7.1 How the rules were deployed.....	17
8. Rule documentation and triggered-alert evidence.....	19
8.1 Rule 100100 - SSH brute force.....	19
8.2 Rule 100101 - repeated sudo escalation to root.....	20
8.3 Rule 100103 - new local account created.....	21
8.4 Rule 100102 - system discovery commands.....	21
9. Alert volume dashboard.....	22
10. MITRE ATT&CK mapping.....	25
11. Reproducing this build.....	26
12. Limitations and what I would do next.....	27

1. What this report covers

This is the write-up for my Week 2 lab task. The goal was to stand up a working SIEM in a lab of my own and prove that I could write detection content for it, rather than just read about detection content. I used Wazuh 4.14.7, deployed from the official OVA, with a Kali Linux virtual machine attached to it as a monitored agent.

By the end I had four custom rules in production on the manager, all four firing on traffic I generated myself, a saved dashboard summarizing alert volume by rule and severity, and every rule mapped to a MITRE ATT&CK technique. The task asked for three rules; I ended up with four because the fourth one (system discovery) was the quickest way to prove my rule file was actually being loaded before I moved on to the harder frequency-based rules.

Everything below is in the order I actually did it, so another analyst should be able to follow it start to finish and end up in the same place.

2. Lab architecture

Both machines run under Oracle VirtualBox on a single Windows host. I put them on a dedicated internal NAT network so the lab traffic stays away from my home LAN, and forwarded only the dashboard port back to the host so I could use the browser on Windows instead of inside the VM.

Component	Detail
SIEM platform	Wazuh 4.14.7 (manager, indexer, dashboard) from the official OVA
Server OS	Amazon Linux based OVA appliance, hostname wazuh-server
Monitored endpoint	Kali Linux 2026.3, agent name Kali-Lab, agent ID 003
Agent version	Wazuh agent 4.14.7-1, amd64 DEB package
Virtualisation	Oracle VirtualBox, NAT Network named Wazuh-network
Host access	Port forward from host loopback to the dashboard on the server
Custom rule file	<code>/var/ossec/etc/rules/custom_rules.xml</code>
Rules written	100100, 100101, 100102, 100103

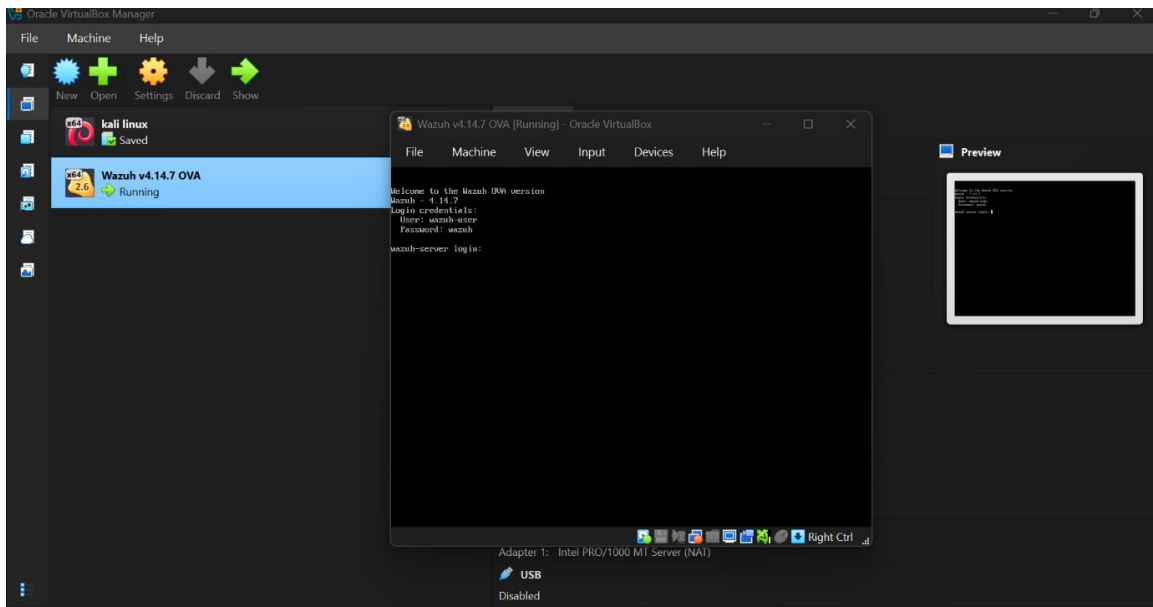


Figure 1 - Both lab machines registered in VirtualBox Manager, with the Wazuh appliance running.

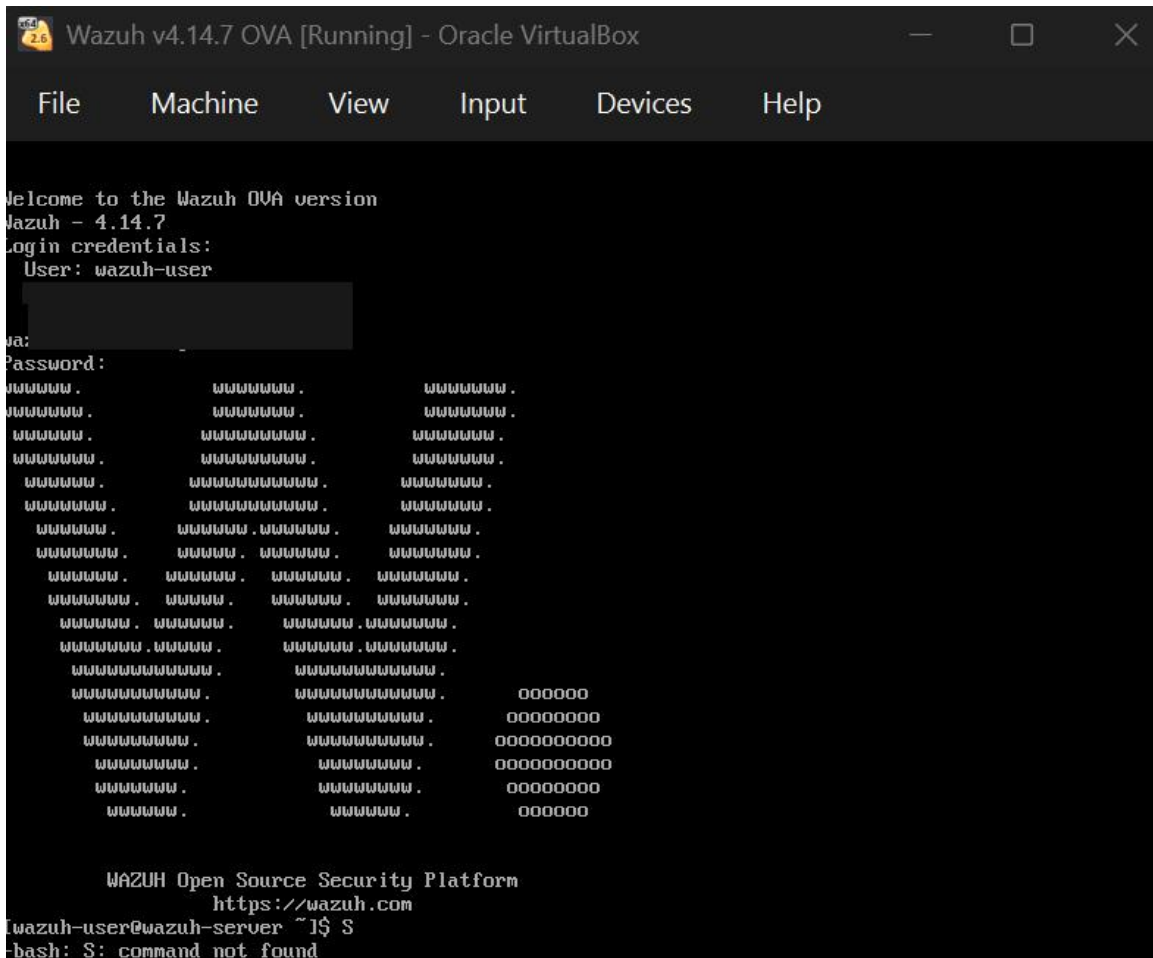


Figure 2 - First boot of the Wazuh OVA. The appliance ships with a default console account, which is covered here.

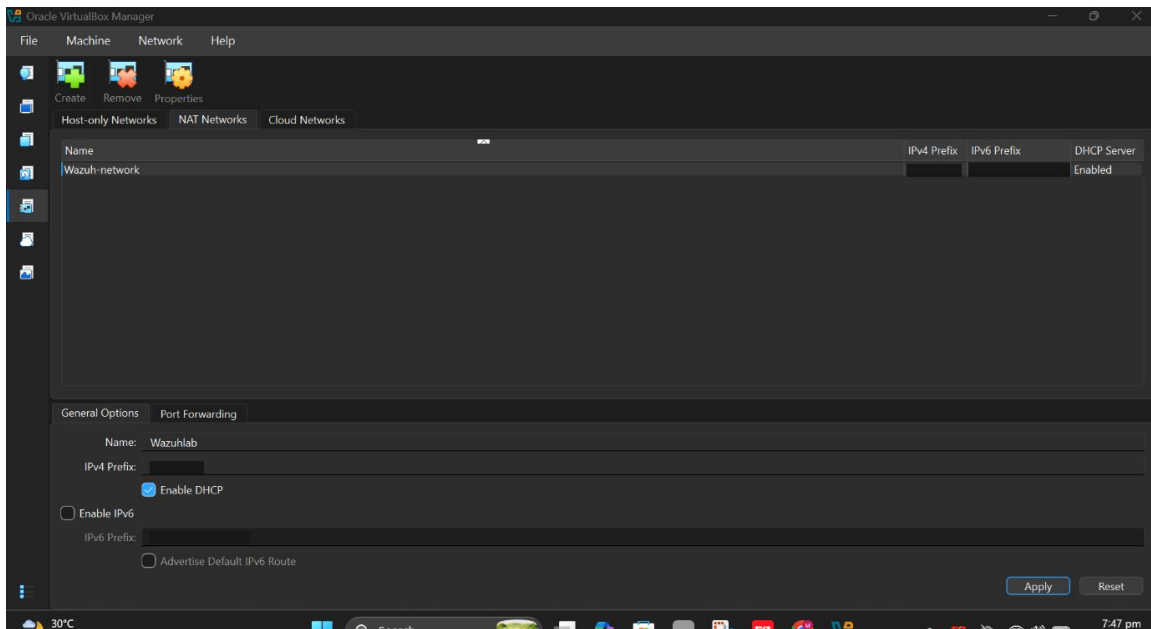


Figure 3 - The dedicated NAT network the two VMs share.

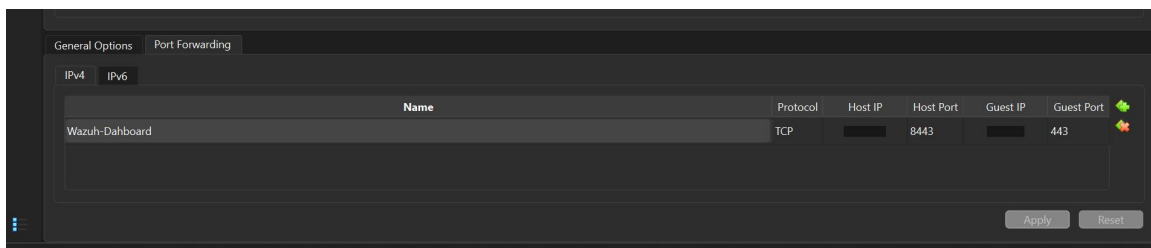


Figure 4 - Port forwarding rule so the dashboard is reachable from the Windows host browser.

3. Bringing the Wazuh server up

Wazuh is three services rather than one, and if any of them is down the symptoms look similar from the browser, so the first thing I did after boot was check all three individually instead of assuming the appliance had come up clean.

```
wazuh-user@wazuh-server ~]$ sudo systemctl status wazuh-manager
wazuh-manager.service - Wazuh manager
Loaded: loaded (/usr/lib/systemd/system/wazuh-manager.service; enabled; preset: disabled)
Active: active (running) since Thu 2026-08-13 12:28:31 UTC; 6min ago
Process: 2053 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0)
Tasks: 228 (limit: 4646)
Memory: 1000.7M
CPU: 3min 39.823s
CGroup: /system.slice/wazuh-manager.service
├─2313 /var/ossec/api/scripts/wazuh_apid.py
├─2337 /var/ossec/api/scripts/wazuh_apid.py
├─2338 /var/ossec/api/scripts/wazuh_apid.py
├─2346 /var/ossec/api/scripts/wazuh_apid.py
├─2352 /var/ossec/api/scripts/wazuh_apid.py
├─2370 /var/ossec/bin/wazuh-authd
├─2407 /var/ossec/bin/wazuh-db
├─2478 /var/ossec/bin/wazuh-execd
├─2502 /var/ossec/bin/wazuh-analysisd
├─2518 /var/ossec/bin/wazuh-logcollector
├─2540 /var/ossec/bin/wazuh-logcollector
├─2557 /var/ossec/bin/wazuh-logcollector
├─2574 /var/ossec/bin/wazuh-logcollector
└─2589 /var/ossec/bin/wazuh-modulesd
```

Figure 5 - wazuh-manager active and running, with the analysis, remoted and logcollector child processes visible.

```

Loaded: loaded (/usr/lib/systemd/system/wazuh-indexer.service; enabled; preset: disabled)
Active: active (running) since Thu 2026-08-13 12:30:55 UTC; 3min 13s ago
Docs: https://documentation.wazuh.com
Main PID: 2052 (java)
Tasks: 94 (limit: 4646)
Memory: 2.3G
CPU: 2min 44.038s
CGroup: /system.slice/wazuh-indexer.service
└─2052 /usr/share/wazuh-indexer/jdk/bin/java -Xshare:auto -Dopensearch.networkaddress.
Aug 13 12:28:24 wazuh-server systemd-entrypoint[2052]: WARNING: A terminally deprecated method in j
Aug 13 12:28:24 wazuh-server systemd-entrypoint[2052]: WARNING: System::setSecurityManager has been
Aug 13 12:28:24 wazuh-server systemd-entrypoint[2052]: WARNING: Please consider reporting this to t
Aug 13 12:28:24 wazuh-server systemd-entrypoint[2052]: WARNING: System::setSecurityManager will be
Aug 13 12:28:30 wazuh-server systemd-entrypoint[2052]: Aug 13, 2026 12:28:29 PM sun.util.locale.pro
Aug 13 12:28:30 wazuh-server systemd-entrypoint[2052]: WARNING: COMPAT locale provider will be remo
Aug 13 12:28:38 wazuh-server systemd-entrypoint[2052]: WARNING: A terminally deprecated method in j
Aug 13 12:28:38 wazuh-server systemd-entrypoint[2052]: WARNING: System::setSecurityManager has been
Aug 13 12:28:38 wazuh-server systemd-entrypoint[2052]: WARNING: Please consider reporting this to t
Aug 13 12:28:38 wazuh-server systemd-entrypoint[2052]: WARNING: System::setSecurityManager will be

```

Figure 6 - wazuh-indexer active. This is the OpenSearch component that stores the alerts.

```
wazuh-user@wazuh-server:~$ sudo systemctl status wazuh-dashboard
wazuh-dashboard.service - wazuh-dashboard
Loaded: loaded (/etc/systemd/system/wazuh-dashboard.service; enabled; preset: disabled)
Active: active (running) since Thu 2026-08-13 12:27:57 UTC; 4min 7s ago
Main PID: 1709 (node)
Tasks: 11 (limit: 4646)
Memory: 185.7M
CPU: 24.585s
CGroup: /system.slice/wazuh-dashboard.service
└─1709 ──no-warnings --max-http-header-size=6

Aug 13 12:31:14 wazuh-server opensearch-dashboards[1709]: {"type":"log","@timestamp":"2026-08-13T12:31:14.000Z","@version":1,"log.level":"info","message":"[SecurityAuthenticator] Authentication successful for user 'admin'."}
Aug 13 12:31:14 wazuh-server opensearch-dashboards[1709]: {"type":"log","@timestamp":"2026-08-13T12:31:14.000Z","@version":1,"log.level":"info","message":"[SecurityAuthenticator] Authentication successful for user 'admin'."}
Aug 13 12:31:15 wazuh-server opensearch-dashboards[1709]: {"type":"log","@timestamp":"2026-08-13T12:31:15.000Z","@version":1,"log.level":"info","message":"[SecurityAuthenticator] Authentication successful for user 'admin'."}
Aug 13 12:31:15 wazuh-server opensearch-dashboards[1709]: {"type":"log","@timestamp":"2026-08-13T12:31:15.000Z","@version":1,"log.level":"info","message":"[SecurityAuthenticator] Authentication successful for user 'admin'."}
Aug 13 12:31:15 wazuh-server opensearch-dashboards[1709]: {"type":"log","@timestamp":"2026-08-13T12:31:15.000Z","@version":1,"log.level":"info","message":"[SecurityAuthenticator] Authentication successful for user 'admin'."}
Aug 13 12:31:17 wazuh-server opensearch-dashboards[1709]: {"type":"log","@timestamp":"2026-08-13T12:31:17.000Z","@version":1,"log.level":"info","message":"[SecurityAuthenticator] Authentication successful for user 'admin'."}
Aug 13 12:31:17 wazuh-server opensearch-dashboards[1709]: {"type":"log","@timestamp":"2026-08-13T12:31:17.000Z","@version":1,"log.level":"info","message":"[SecurityAuthenticator] Authentication successful for user 'admin'."}
Aug 13 12:31:17 wazuh-server opensearch-dashboards[1709]: {"type":"log","@timestamp":"2026-08-13T12:31:17.000Z","@version":1,"log.level":"info","message":"[SecurityAuthenticator] Authentication successful for user 'admin'."}
Aug 13 12:31:19 wazuh-server opensearch-dashboards[1709]: {"type":"log","@timestamp":"2026-08-13T12:31:19.000Z","@version":1,"log.level":"info","message":"[SecurityAuthenticator] Authentication successful for user 'admin'."}
Aug 13 12:31:21 wazuh-server opensearch-dashboards[1709]: {"type":"log","@timestamp":"2026-08-13T12:31:21.000Z","@version":1,"log.level":"info","message":"[SecurityAuthenticator] Authentication successful for user 'admin'."}
lines 1-20/20 (END)
```

Figure 7 - wazuh-dashboard active.

```
Aug 13 12:28:28 wazuh-server env[2053]: wazuh-logcollector: Process 2731 not used by Wazuh, removing...
Aug 13 12:28:28 wazuh-server env[2053]: Started wazuh-logcollector...
Aug 13 12:28:28 wazuh-server env[2053]: wazuh-monitord: Process 2748 not used by Wazuh, removing...
Aug 13 12:28:28 wazuh-server env[2053]: Started wazuh-monitord...
Aug 13 12:28:28 wazuh-server env[2053]: wazuh-modulesd: Process 2758 not used by Wazuh, removing...
Aug 13 12:28:28 wazuh-server env[2586]: 2026/08/13 12:28:28 wazuh-modulesd:router: INFO: Loaded router...
Aug 13 12:28:28 wazuh-server env[2586]: 2026/08/13 12:28:28 wazuh-modulesd:content_manager: INFO: Loaded...
Aug 13 12:28:28 wazuh-server env[2586]: 2026/08/13 12:28:28 wazuh-modulesd:inventory-harvester: INFO: Loaded...
Aug 13 12:28:29 wazuh-server env[2053]: Started wazuh-modulesd...
Aug 13 12:28:31 wazuh-server env[2053]: Completed.
[wazuh-user@wazuh-server ~]$ curl -k https://localhost:5555/
```

Figure 8 - Querying the indexer directly on its API port. The unauthorized response is expected without credentials and confirms the service is listening.

```

https://wazuh.com
[wazuh-user@wazuh-server ~]$ S
-bash: S: command not found
[wazuh-user@wazuh-server ~]$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:00:00:00 brd ff:ff:ff:ff:ff:ff
    altname enp0s17
    inet 10.10.10.24 metric 1024 brd 10.10.10.255 scope global dynamic eth0
        valid_lft 86182sec preferred_lft 86182sec
    inet6 fe80::208:10ff:fe00:0024 scope global dynamic mngtmpaddr noprefixroute
        valid_lft 86185sec preferred_lft 14185sec
    inet6 fe80::208:10ff:fe00:0024 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
[wazuh-user@wazuh-server ~]$

```

Figure 9 - Interface configuration on the server, used to confirm which address the agent should point at.

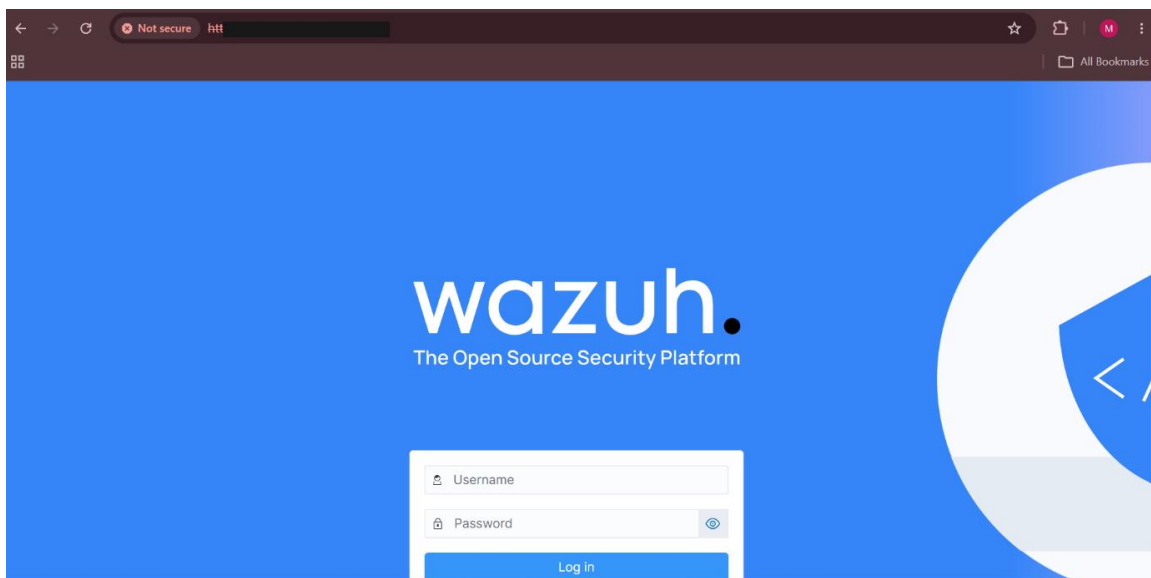


Figure 10 - The dashboard login page reached from the host browser through the forwarded port.

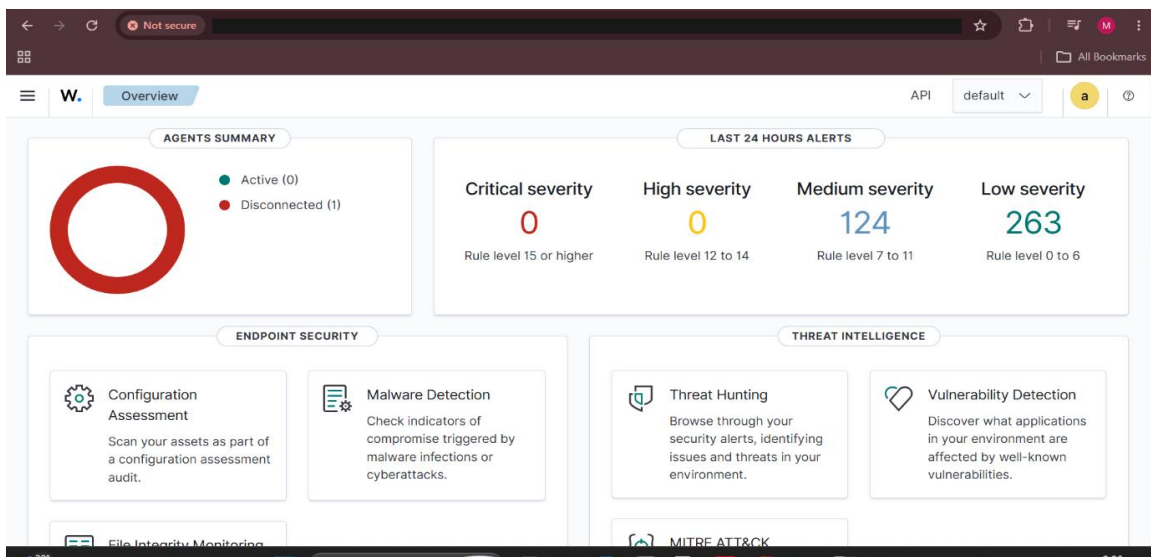


Figure 11 - Wazuh overview after login. At this stage no agent is connected, which is why the agent summary shows disconnected.

4. Network path between the two machines

The agent talks to the manager on 1514/TCP and enrolls on 1515/TCP. Rather than deploy the agent and then debug a silent failure, I proved the path in both directions first. This saved time later: when the agent did come up it connected on the first attempt.

```
[wazuh-user@wazuh-server ~]$ hostname -I
[redacted]
[wazuh-user@wazuh-server ~]$ sudo ss -tulpn | grep 443
tcp LISTEN 0      511      [redacted]      [redacted]      users:((("node",pid=1688,fd=19)
)
[wazuh-user@wazuh-server ~]$ ping -c 4 [redacted]
PING [redacted] 56(84) bytes of data.
64 bytes from [redacted]: icmp_seq=1 ttl=64 time=5.30 ms
64 bytes from [redacted]: icmp_seq=2 ttl=64 time=2.78 ms
64 bytes from [redacted]: icmp_seq=3 ttl=64 time=1.51 ms
64 bytes from [redacted]: icmp_seq=4 ttl=64 time=1.30 ms

--- [redacted] ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3016ms
rtt min/avg/max/mdev = 1.296/2.721/5.303/1.594 ms
[wazuh-user@wazuh-server ~]$
```

Figure 12 - On the server: confirming the dashboard port is bound and that the Kali machine answers ICMP.

```
(kali@kali)~$ ip address
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000 link/loopback [redacted] brd [redacted]
    inet [redacted] scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000 link/ether [redacted] brd ff:ff:ff:ff:ff:ff
    inet [redacted] scope global dynamic noprefixroute eth0
        valid_lft 563sec preferred_lft 563sec
    inet6 [redacted] scope link noprefixroute
        valid_lft forever preferred_lft forever

(kali@kali)~$ ip route
default via [redacted] dev eth0 proto dhcp src [redacted] metric 100
[redacted] dev eth0 proto kernel scope link src [redacted] metric 100

(kali@kali)~$ ip route
default via [redacted] dev eth0 proto dhcp src [redacted] metric 100
[redacted] dev eth0 proto kernel scope link src [redacted] metric 100

(kali@kali)~$ hostname -I
[redacted]

(kali@kali)~$ ip route
default via [redacted] dev eth0 proto dhcp src [redacted] metric 100
[redacted] dev eth0 proto kernel scope link src [redacted] metric 100

(kali@kali)~$ ping -c 4 [redacted]
PING [redacted] 56(84) bytes of data.
64 bytes from [redacted]: icmp_seq=1 ttl=127 time=9.97 ms
64 bytes from [redacted]: icmp_seq=2 ttl=127 time=1.16 ms
64 bytes from [redacted]: icmp_seq=3 ttl=127 time=1.60 ms
64 bytes from [redacted]: icmp_seq=4 ttl=127 time=61.6 ms

--- [redacted] ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 1.160/18.591/61.638/25.099 ms

(kali@kali)~$ nc -zv [redacted] 1514
[redacted] inverse host lookup failed: Unknown host
```

Figure 13 - On Kali: interface and routing configuration, and a successful ping back to the server.

```

(kali@kali)-[~]
$ nc -zv [redacted] 1515
Connection to [redacted] 1515 port [tcp/*] succeeded!

(kali@kali)-[~]
$ nc -zv [redacted] 1514
Connection to [redacted] 1514 port [tcp/*] succeeded!

(kali@kali)-[~]
$
```

Figure 14 - Both agent ports reachable from Kali. This is the check that matters before installing the agent.

5. Deploying and enrolling the agent

I used the dashboard's deployment helper to generate the install command for a Debian based, 64-bit endpoint, then ran it on Kali. I also registered the agent manually on the server side with `manage_agents` so I could see the enrolment from both ends.

Two small things went wrong here and both are visible in the screenshots. The appliance does not ship with `ufw`, so my first firewall check returned `command not found`, and I mistyped the `manage_agents` binary the first time. Neither affected the outcome but I have left them in rather than retaking the screenshots, because the working commands are on the same screen.

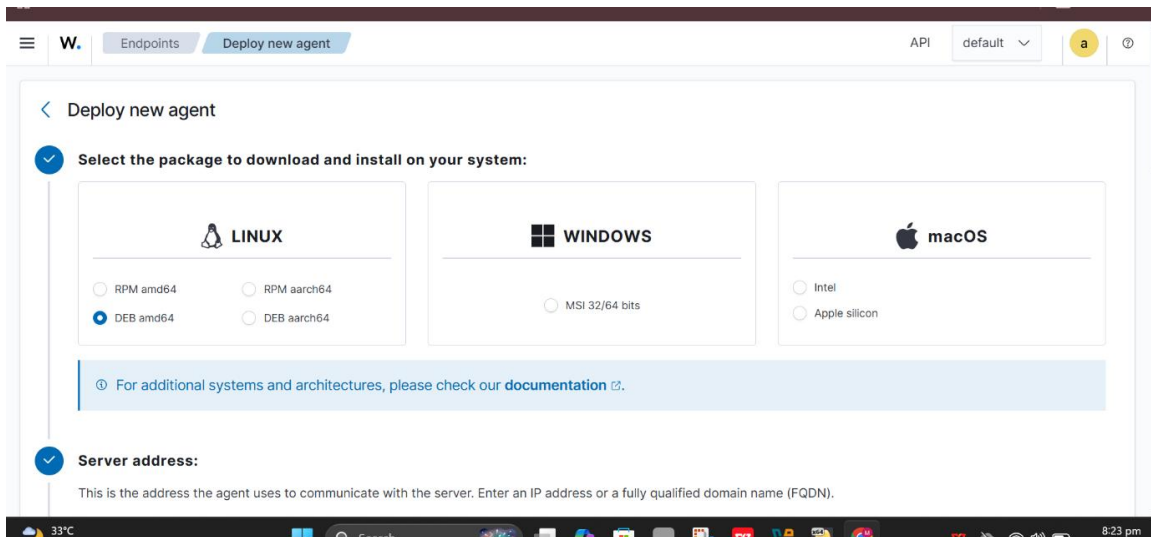


Figure 15 - Generating the agent deployment command from the dashboard.

```

[wazuh-user@wazuh-server ~]$ sudo /var/ossec/bin/manage_agents
sudo: /var/ossec/bin/manage_agents: command not found
[wazuh-user@wazuh-server ~]$ sudo /var/ossec/bin/manage_agents

*****
* Wazuh v4.14.7 Agent manager.                *
* The following options are available: *
*****
(A)dd an agent (A).
(E)xtract key for an agent (E).
(L)ist already added agents (L).
(R)emove an agent (R).
(Q)uit.
Choose your action: A,E,L,R or Q:

```

Figure 16 - The manage_agents menu on the server.

```

[ua00:27ff-fea3:5660/64]
[wazuh-user@wazuh-server ~]$ sudo ufw status
sudo: ufw: command not found
[wazuh-user@wazuh-server ~]$ sudo /var/ossec/bin/manage_agents
sudo: /var/ossec/bin/manage_agents: command not found
[wazuh-user@wazuh-server ~]$ sudo /var/ossec/bin/manage_agents

*****
* Wazuh v4.14.7 Agent manager.                *
* The following options are available: *
*****
(A)dd an agent (A).
(E)xtract key for an agent (E).
(L)ist already added agents (L).
(R)emove an agent (R).
(Q)uit.
Choose your action: A,E,L,R or Q: A

- Adding a new agent (use '\q' to return to the main menu).
  Please provide the following:
    * A name for the new agent: kali-linux
    * The IP Address of the new agent: 
Confirm adding it?(y/n): y
Agent added with ID 001.

*****
* Wazuh v4.14.7 Agent manager.                *
* The following options are available: *
*****
(A)dd an agent (A).
(E)xtract key for an agent (E).
(L)ist already added agents (L).
(R)emove an agent (R).
(Q)uit.
Choose your action: A,E,L,R or Q:

```

Figure 17 - Registering the endpoint and receiving agent ID 001 from this registration path.

```
Session Actions Edit View Help
GNU nano 9.1
Wazuh - Agent - Default configuration for kali 2026.3
More info at: https://documentation.wazuh.com
Mailing list: https://groups.google.com/forum/#!forum/wazuh

<ossec_config>
  <client>
    <server>
      <address>[redacted]/address>
      <port>1514</port>
      <protocol>[redacted]otocol>
    </server>
    <config-profile>kali, kali2026, kali2026.3</config-profile>
    <notify_time>20</notify_time>
    <time-reconnect>60</time-reconnect>
    <auto_restart>yes</auto_restart>
    <crypto_method>aes</crypto_method>
    <enrollment>
      <enabled>yes</enabled>
      <agent_name>Kali-Lab</agent_name>
    </enrollment>
  </client>

  <client_buffer>
    <disabled>no</disabled>
    <queue_size>5000</queue_size>
    <events_per_second>500</events_per_second>
  </client_buffer>

  <Policy monitoring ->
  <rootcheck>
    <disabled>no</disabled>
    <check_files>yes</check_files>
    <check_trojans>yes</check_trojans>
    <check_dev>yes</check_dev>
    <check_sys>yes</check_sys>
    <check_pids>yes</check_pids>
```

Figure 18 - The agent configuration file on Kali, showing the manager address, the enrolment block and the agent name Kali-Lab.

```
(kali@kali)-[~]
$ dpkg -l | grep wazuh-agent
ii wazuh-agent 4.14.7-1 amd64 Wazuh agent

(kali@kali)-[~]
$ sudo systemctl status wazuh-agent
● wazuh-agent.service - Wazuh agent
   Loaded: loaded (/usr/lib/systemd/system/wazuh-agent.service; enabled; preset: disabled)
   Active: active (running) since Thu 2026-08-13 12:23:34 EDT; 39min ago
     Invocation:
        Tasks: 28 (limit: 3486)
        Memory: 48.1M (peak: 71.8M)
        CPU: 11.367s
        CGroup: /system.slice/wazuh-agent.service
                └─17830 /var/ossec/bin/wazuh-execd
                  17838 /var/ossec/bin/wazuh-agentd
                  17859 /var/ossec/bin/wazuh-syscheckd
                  17869 /var/ossec/bin/wazuh-logcollector
                  17895 /var/ossec/bin/wazuh-modulesd

Aug 13 12:23:28 kali systemd[1]: Starting wazuh-agent.service - Wazuh agent...
Aug 13 12:23:28 kali env[17808]: Starting Wazuh v4.14.7 ...
Aug 13 12:23:28 kali env[17808]: Started wazuh-execd ...
Aug 13 12:23:29 kali env[17808]: Started wazuh-agentd ...
Aug 13 12:23:29 kali env[17808]: Started wazuh-syscheckd ...
Aug 13 12:23:30 kali env[17808]: Started wazuh-logcollector ...
Aug 13 12:23:31 kali env[17808]: Started wazuh-modulesd ...
Aug 13 12:23:33 kali env[17808]: Completed.
Aug 13 12:23:34 kali systemd[1]: Started wazuh-agent.service - Wazuh agent.

(kali@kali)-[~]
$ sudo /var/ossec/bin/wazuh-control status
wazuh-modulesd is running...
wazuh-logcollector is running...
wazuh-syscheckd is running...
wazuh-agentd is running...
wazuh-execd is running...
```

Figure 19 - The agent service running on Kali, with all five agent daemons started.

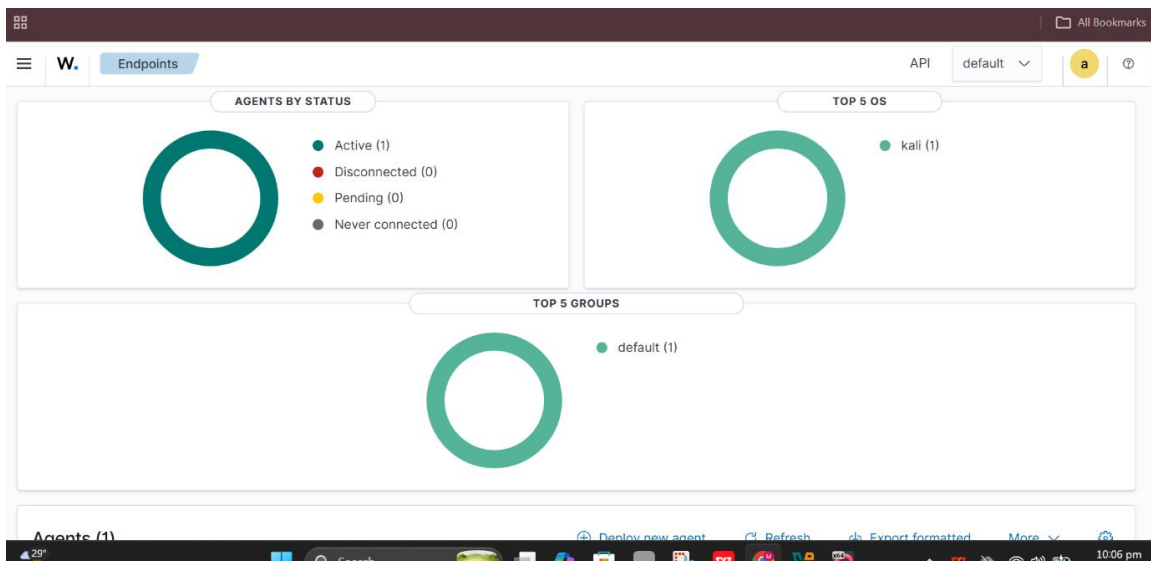


Figure 20 - The manager now reports one active agent running Kali. This satisfies the connected-endpoint requirement.

6. Generating baseline events before writing any rules

Before writing detection content I wanted to confirm that authentication events were actually reaching the indexer, because a rule that never fires is indistinguishable from a rule that was never loaded. SSH is not enabled by default on Kali, so I started and enabled the service first, then failed a login against it on purpose.

```
(kali@kali)-[~]
└─$ sudo systemctl status ssh
[sudo] password for kali:
o ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: disabled)
   Active: inactive (dead)
     Docs: man:sshd(8)
           man:sshd_config(5)

(kali@kali)-[~]
└─$ sudo systemctl start ssh

(kali@kali)-[~]
└─$ sudo systemctl status ssh --no-pager
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: disabled)
   Active: active (running) since Fri 2026-08-14 13:15:39 EDT; 9s ago
     Invocation:
       Docs: man:sshd(8)
             man:sshd_config(5)
    Process: 86614 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Main PID: 86625 (sshd)
      Tasks: 1 (limit: 3486)
     Memory: 1.8M (peak: 2.6M)
        CPU: 125ms
    CGroup: /system.slice/ssh.service
            └─86625 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Aug 14 13:15:39 kali systemd[1]: Starting ssh.service - OpenBSD Secure Shell server...
Aug 14 13:15:39 kali sshd[86625]: Server listening on port 22.
Aug 14 13:15:39 kali sshd[86625]: Server listening on :: port 22.
Aug 14 13:15:39 kali systemd[1]: Started ssh.service - OpenBSD Secure Shell server.

(kali@kali)-[~]
└─$ sudo systemctl enable ssh
Created symlink '/etc/systemd/system/ssh.service' → '/usr/lib/systemd/system/ssh.service'.
Created symlink                                → '/usr/lib/systemd/system/ssh.service'.

(kali@kali)-[~]
└─$ sudo ss -ltnp | grep ':22'
LISTEN 0      128          *:*                users:((("sshd",pid=86625,fd=6))
LISTEN 0      128          [::]:*            users:((("sshd",pid=86625,fd=7))
```

Figure 21 - Enabling and starting the SSH service on the agent, and confirming it is listening.

```
(kali@kali)-[~]
$ sudo systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: disabled)
   Active: active (running) since Fri 2026-08-14 13:15:39 EDT; 58s ago
     Invocation:
       Docs: man:sshd(8)
             man:sshd_config(5)
    Main PID: 86625 (sshd)
      Tasks: 1 (limit: 3486)
     Memory: 1.5M (peak: 2.6M)
        CPU: 125ms
     CGroup: /system.slice/ssh.service
            └─86625 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Aug 14 13:15:39 kali systemd[1]: Starting ssh.service - OpenBSD Secure Shell server ...
Aug 14 13:15:39 kali sshd[86625]: Server listening on port 22.
Aug 14 13:15:39 kali sshd[86625]: Server listening on :: port 22.
Aug 14 13:15:39 kali systemd[1]: Started ssh.service - OpenBSD Secure Shell server.

(kali@kali)-[~]
$ ssh wronguser@localhost
The authenticity of host 'localhost (::1)' can't be established.
ED25519 key fingerprint is:
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'localhost' (ED25519) to the list of known hosts.
wronguser@localhost's
Connection closed by ::1 port 22

(kali@kali)-[~]
$
```

Figure 22 - Service status after enabling, followed by a deliberate failed login as a non-existent user.

```
(kali@kali)-[~]
$ ssh wronguser@localhost
wronguser@localhost's
Permission denied, please try again.
wronguser@localhost's
Permission denied, please try again.
wronguser@localhost's
wronguser@localhost: Permission denied (publickey,password).
```

Figure 23 - Three consecutive failed password attempts, which is the behaviour the brute-force rule needs to see.

Those events came through immediately, which told me the log pipeline was healthy and I could start writing rules against it.

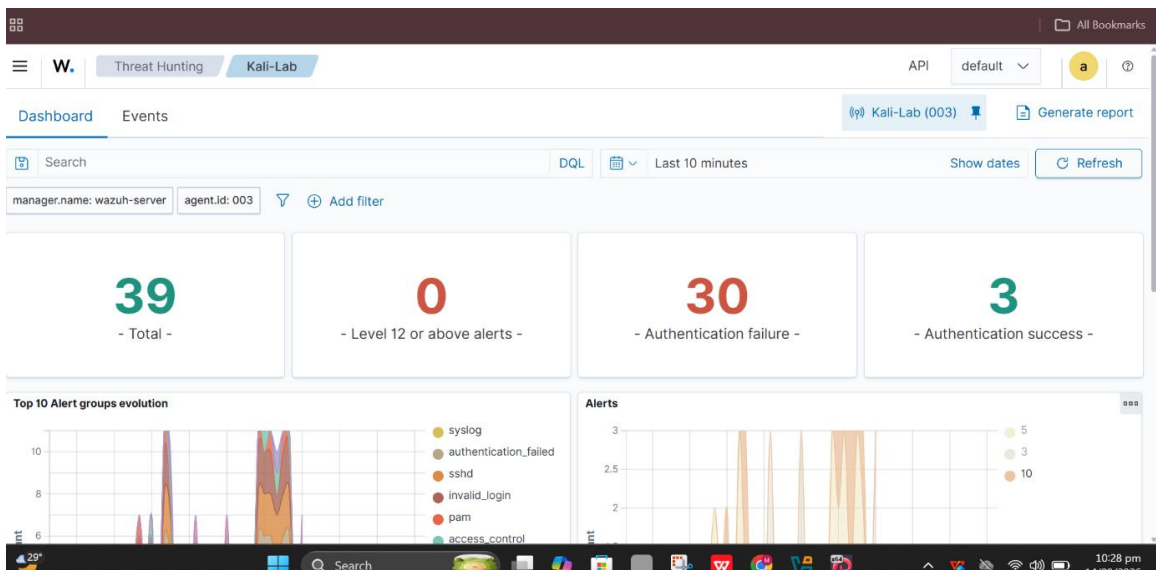


Figure 24 - Threat hunting view filtered to the agent, showing 30 authentication failures against 3 successes.

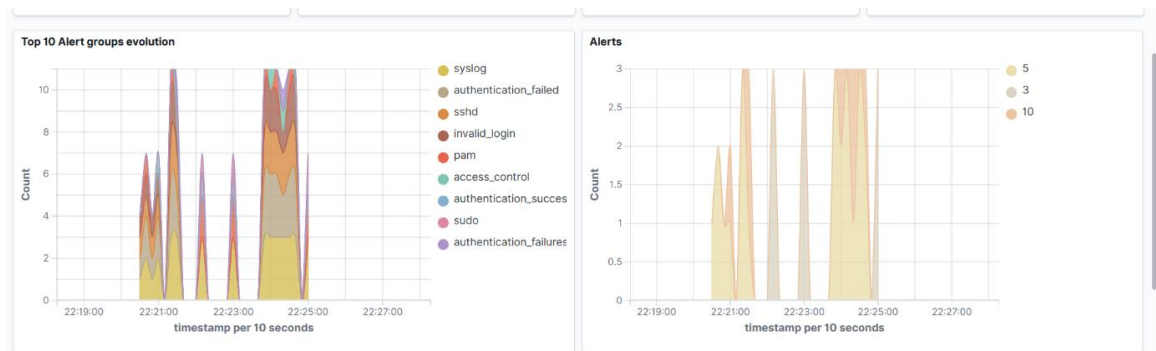


Figure 25 - Alert groups over time. The clusters line up with the times I was generating failed logins.

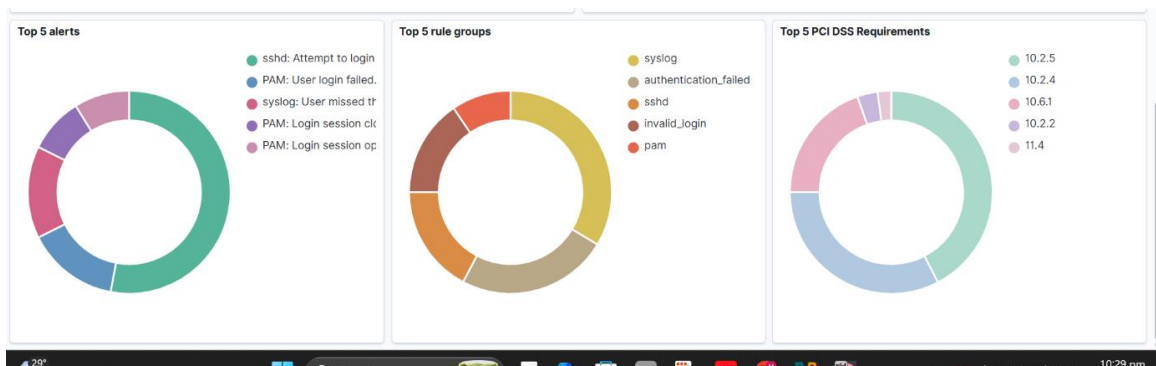


Figure 26 - Built-in breakdown of top alerts, rule groups and PCI DSS requirements for the generated traffic.

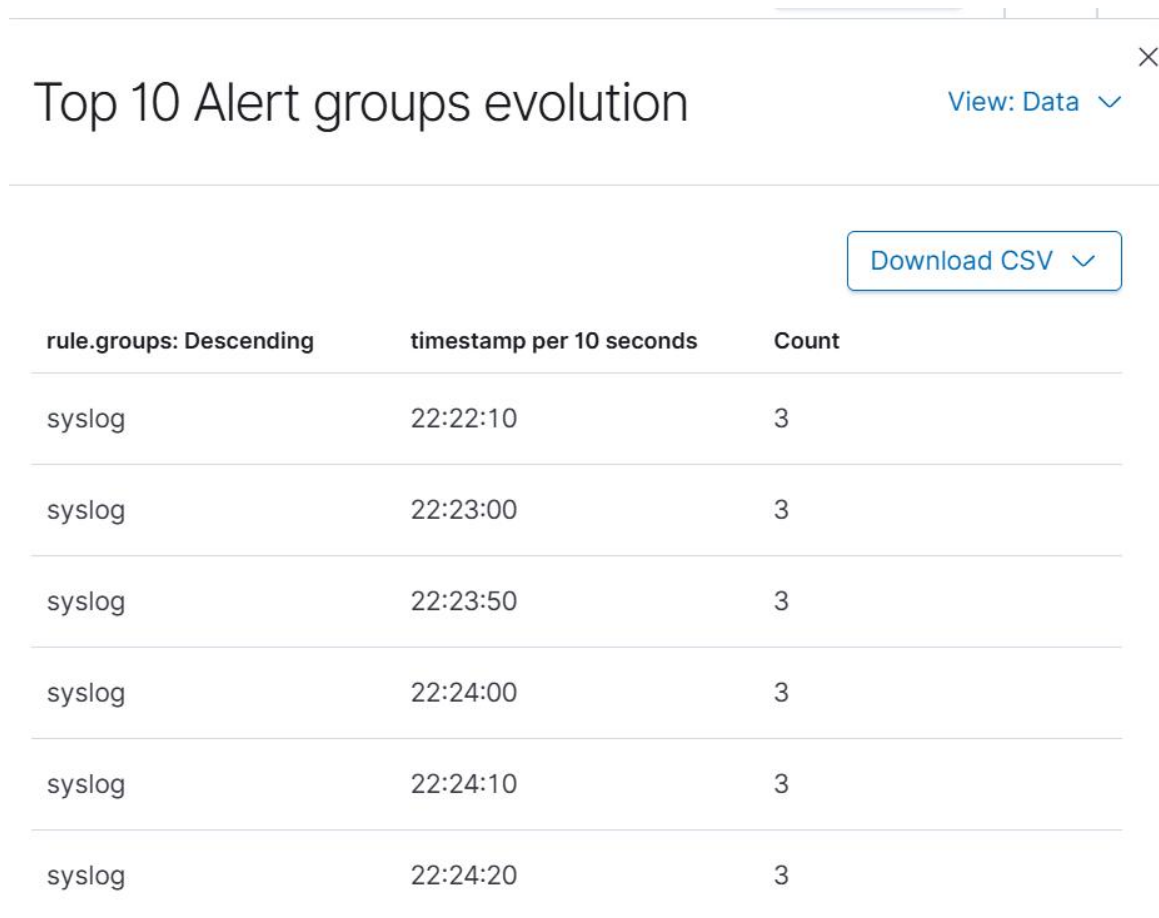


Figure 27 - The same data inspected as a table, which is how I confirmed the event counts rather than reading them off a chart.

7. Writing the detection rules

Wazuh ships a `local_rules.xml` with a worked example in it. I read that first to get the structure right, then wrote my own rules into a separate `custom_rules.xml` file so my content stayed separate from anything supplied by the vendor and could be lifted out on its own.

```

GNU nano 8.3 /var/ossec/etc/rules/local_rules.xml
<!-- Local rules -->

<!-- Modify it at your will. -->
<!-- Copyright (C) 2015, Wazuh Inc. -->

<!-- Example -->
<group name="local,syslog,sshd,">

  <!--
  Dec 10 01:02:02 host sshd[1234]: Failed none for root from [REDACTED] port 1066 ssh2
  -->
  <rule id="100001" level="5">
    <if_sid>5716</if_sid>
    [REDACTED]
    <description>sshd: authentication failed from IP [REDACTED]</description>
    <group>authentication_failed,pci_dss_10.2.4,pci_dss_10.2.5,</group>
  </rule>

</group>

^G Help      ^O Write Out  ^F Where Is   [ Read 19 lines ] ^T Execute    ^C Location   ^U Undo
^X Exit      ^R Read File  ^_ Replace    ^K Cut        ^J Justify    ^G Go To Line ^E Redo

```

Figure 28 - The stock local rules file, used as a reference for rule structure.

I started with a single rule in its own group to prove the loading mechanism worked end to end before adding the rest.

```

GNU nano 9.1
<group name="custom_ssh,">
  <rule id="100100" level="12" frequency="5" timeframe="60">
    <if_matched_sid>5710</if_matched_sid>
    <description>CUSTOM: SSH Brute Force - Multiple Invalid User Login Attempts</description>
    <mitre>
      <id>T1110</id>
    </mitre>
  </rule>
</group>

```

Figure 29 - The first rule written on its own, before the remaining three were added.

The finished rule file contains all four rules. Two of them are frequency based, meaning they only fire when the underlying event repeats within a time window, and two are single event rules.

```

GNU nano 9.1
group name="custom_rules,">
  <!-- Rule 1: SSH brute force -->
  <rule id="100100" level="12" frequency="5" timeframe="60">
    <if_matched_sid>5710</if_matched_sid>
    <description>CUSTOM: SSH Brute Force - Multiple Invalid User Login Attempts</description>
    <mitre>
      <id>T1110</id>
    </mitre>
  </rule>

  <!-- Rule 2: Multiple sudo/root executions -->
  <rule id="100101" level="10" frequency="3" timeframe="60">
    <if_matched_sid>5402</if_matched_sid>
    <description>CUSTOM: Multiple Successful Sudo to ROOT Executions</description>
    <mitre>
      <id>T1548.003</id>
    </mitre>
  </rule>

  <!-- Rule 3: System discovery -->
  <rule id="100102" level="8">
    <match>whoami|uname|hostname</match>
    <description>CUSTOM: Suspicious System Discovery Command Executed</description>
    <mitre>
      <id>T1033</id>
    </mitre>
  </rule>

  <!-- Rule 4: User creation -->
  <rule id="100103" level="12">
    <if_sid>5902</if_sid>
    <match>useradd</match>
    <description>CUSTOM: New User Account Creation Detected</description>
    <mitre>
      <id>T1136.001</id>
    </mitre>
  </rule>
</group>

```

Figure 30 - The complete custom rule file with all four rules and their ATT&CK mappings.

7.1 How the rules were deployed

Rather than editing the file directly on the manager, I pushed it from the agent machine using the Wazuh REST API. This is closer to how rule content is managed in a real environment, where analysts do not usually have a shell on the SIEM itself. The API requires a bearer token, which is requested with the API user credentials and is valid for a short period. Every token in the screenshots below has been covered.

```

(kali@kali)-[~]
$ curl -k -u [redacted] \
-X POST \
[redacted]
e

(kali@kali)-[~]
$ echo $TOKEN
*** More

(kali@kali)-[~]
$ TOKEN=$(curl -s -k -u [redacted] \
-X POST \
[redacted]

```

Figure 31 - Requesting an API token and storing it in a shell variable.

```
(kali@kali)-[~]
$ curl -k \

-H "Authorization: Bearer $TOKEN"
{"data": {"title": "Wazuh API REST", "api_version": "4.6.0", "error": 0}}
```

Figure 32 - A test call against the API root to confirm the token is accepted.

```
(kali@kali)-[~]
$ nano custom_rules.xml

(kali@kali)-[~]
$ ls -l custom_rules.xml
-rw-rw-r-- 1 kali kali 191 Aug 16 21:24 custom_rules.xml

(kali@kali)-[~]
$ nano custom_rules.xml

(kali@kali)-[~]
$ curl -sk \
-X PUT \
"https://$WAZUH_IP:55000/rules/files/custom_rules.xml" \
-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/octet-stream" \
--data-binary @custom_rules.xml
```

Figure 33 - Editing the local copy of the rule file and preparing the upload request.

```
(kali@kali)-[~]
$ TOKEN=$(curl -sk -u ... \
-X PUT \
"ht...
(kali@kali)-[~]
$ echo "$TOKEN" | cut -c1-20

(kali@kali)-[~]
$ curl -sk \
-X PUT \
"ht...
-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/octet-stream" \
--data-binary @custom_rules.xml
{"data": {"affected_items": ["/etc/rules/custom_rules.xml"], "total_affected_items": 1, "total_failed_items": 0, "failed_items": [], "message": "Rule was successfully uploaded", "error": 0}}
```

Figure 34 - The upload succeeding. The response confirms one affected item and no failures.

```
(kali@kali)-[~]
$ curl -sk \
-X GET \
"https://
-H "Authorization: bearer token <token> | grep -o id- [0-9]*"
id="100100"
id="100101"
id="100102"
id="100103"
(kali@kali)-[~]
```

Figure 35 - Reading the file back through the API and extracting the rule IDs, which confirms all four arrived intact.

```
[wazuh-user@wazuh-server ~]$ sudo grep -R "100101" /var/ossec/etc/rules/
[wazuh-user@wazuh-server ~]$ sudo grep -R "100101" /var/ossec/etc/rules/
/var/ossec/etc/rules/custom_rules.xml: <rule id="100101" level="10" frequency="3" timeframe="60">
[wazuh-user@wazuh-server ~]$ sudo grep -R "100102" /var/ossec/etc/rules/
/var/ossec/etc/rules/custom_rules.xml: <rule id="100102" level="8">
[wazuh-user@wazuh-server ~]$ sudo grep -R "100102" /var/ossec/etc/rules/
/var/ossec/etc/rules/custom_rules.xml: <rule id="100102" level="8">
[wazuh-user@wazuh-server ~]$ sudo grep -R "100103" /var/ossec/etc/rules/
/var/ossec/etc/rules/custom_rules.xml: <rule id="100103" level="12">
```

Figure 36 - The same four rules confirmed on the manager filesystem, with their levels, frequencies and timeframes.

8. Rule documentation and triggered-alert evidence

This section covers each rule in turn: what it detects, why an analyst should care about it, and a screenshot of the alert it produced when I generated the matching behaviour.

8.1 Rule 100100 - SSH brute force

Property	Value
Rule ID	100100
Severity level	12
Logic	Fires when rule 5710 matches 5 times within 60 seconds
Parent event	5710 - attempted login using a non-existent user
ATT&CK technique	T1110 - Brute Force (Credential Access)

What it detects: repeated SSH login attempts for accounts that do not exist on the host, arriving faster than a human typing a password wrong would produce.

Why it matters: a single failed SSH login is background noise on any internet-facing host and is not worth an analyst's time. Five in a minute is a different signal, because it suggests something is working through a username list. Catching this at the attempt stage is what gives a SOC the chance to block a source before a valid credential is found. I set the threshold at five rather than three so that my own mistyped logins would not raise an alert.

Export Formatted	Reset view	639 available fields	Columns	Density	1 fields sorted	Full screen
timestamp	agent.name	rule.description	rule.level	rule.id		
Aug 17, 2026 @ [REDACTED]	Kali-Lab	sshd: Attempt to login using a non-existent user	5	5710		
Aug 17, 2026 @ [REDACTED]	Kali-Lab	sshd: Attempt to login using a non-existent user	5	5710		
Aug 17, 2026 @ [REDACTED]	Kali-Lab	sshd: Attempt to login using a non-existent user	5	5710		
Aug 17, 2026 @ [REDACTED]	Kali-Lab	sshd: Attempt to login using a non-existent user	5	5710		
Aug 17, 2026 @ [REDACTED]	Kali-Lab	sshd: Attempt to login using a non-existent user	5	5710		
Aug 17, 2026 @ [REDACTED]	Kali-Lab	sshd: Attempt to login using a non-existent user	5	5710		
Aug 17, 2026 @ [REDACTED]	Kali-Lab	sshd: Attempt to login using a non-existent user	5	5710		
Aug 17, 2026 @ [REDACTED]	Kali-Lab	sshd: Attempt to login using a non-existent user	5	5710		
Aug 17, 2026 @ [REDACTED]	Kali-Lab	sshd: Attempt to login using a non-existent user	5	5710		

Figure 37 - The underlying level 5 events from the failed login attempts.

Export Formatted	Reset view	639 available fields	Columns	Density	1 fields sorted	Full screen
timestamp	agent.name	rule.description	rule.level	rule.id		
Aug 17, 2026 @ [REDACTED]	Kali-Lab	CUSTOM: SSH Brute Force - Multiple Invalid User Login Attempts	12	100100		
Aug 17, 2026 @ [REDACTED]	Kali-Lab	CUSTOM: SSH Brute Force - Multiple Invalid User Login Attempts	12	100100		
Aug 17, 2026 @ [REDACTED]	Kali-Lab	CUSTOM: SSH Brute Force - Multiple Invalid User Login Attempts	12	100100		
Aug 17, 2026 @ [REDACTED]	Kali-Lab	CUSTOM: SSH Brute Force - Multiple Invalid User Login Attempts	12	100100		

Figure 38 - Rule 100100 firing at level 12 on the generated brute-force attempt.

Aug 17, 2026 @ [REDACTED]	Kali-Lab	syslog: User missed the password more than one time	10	2502
Aug 17, 2026 @ [REDACTED]	Kali-Lab	CUSTOM: SSH Brute Force - Multiple Invalid User Login Attempts	12	100100
Aug 17, 2026 @ [REDACTED]	Kali-Lab	sshd: Attempt to login using a non-existent user	5	5710

Figure 39 - The custom alert alongside the built-in alerts from the same burst, showing how the correlation sits above the raw events.

8.2 Rule 100101 - repeated sudo escalation to root

Property	Value
Rule ID	100101
Severity level	10
Logic	Fires when rule 5402 matches 3 times within 60 seconds
Parent event	5402 - successful sudo to ROOT executed
ATT&CK technique	T1548.003 - Sudo and Sudo Caching (Privilege Escalation)

What it detects: the same account escalating to root repeatedly in a short window.

Why it matters: sudo on its own is completely normal, which is exactly why it is useful to an attacker who already has a foothold. The value here is in the rate. An administrator doing maintenance escalates a few times over an afternoon; a script running post-compromise actions escalates several times in under a minute. This rule is deliberately set at level 10 rather than 12 because it is the noisiest of the four and is better treated as something to review than something to page on.

timestamp	agent.name	rule.description	rule.level	rule.id
Aug 17, 2026 @ [redacted]	Kali-Lab	CUSTOM: Multiple Successful Sudo to ROOT Executions	10	100101
Aug 17, 2026 @ [redacted]	Kali-Lab	CUSTOM: Multiple Successful Sudo to ROOT Executions	10	100101

Figure 40 - Rule 100101 firing after repeated escalations.

timestamp	agent.name	rule.mitre.id	rule.mitre.tactic	rule.description	rule.level	rule.id
Aug 17, 2026 @ 11:00:46.8...	Kali-Lab	T1548.003	Privilege Escalation, Defense Evasion	CUSTOM: Multiple Successful Sudo to ROOT Executions	10	100101
Aug 17, 2026 @ [redacted]	Kali-Lab	T1548.003	Privilege Escalation, Defense Evasion	CUSTOM: Multiple Successful Sudo to ROOT Executions	10	100101
Aug 17, 2026 @ [redacted]	Kali-Lab	T1548.003	Privilege Escalation, Defense Evasion	CUSTOM: Multiple Successful Sudo to ROOT Executions	10	100101
Aug 17, 2026 @ [redacted]	Kali-Lab	T1548.003	Privilege Escalation, Defense Evasion	CUSTOM: Multiple Successful Sudo to ROOT Executions	10	100101
Aug 17, 2026 @ [redacted]	Kali-Lab	T1548.003	Privilege Escalation, Defense Evasion	CUSTOM: Multiple Successful Sudo to ROOT Executions	10	100101
Aug 17, 2026 @ 10:59:32.7...	Kali-Lab	T1548.003	Privilege Escalation, Defense Evasion	CUSTOM: Multiple Successful Sudo to ROOT Executions	10	100101
Aug 17, 2026 @ [redacted]	Kali-Lab	T1548.003	Privilege Escalation, Defense Evasion	CUSTOM: Multiple Successful Sudo to ROOT Executions	10	100101
Aug 17, 2026 @ [redacted]	Kali-Lab	T1548.003	Privilege Escalation, Defense Evasion	CUSTOM: Multiple Successful Sudo to ROOT Executions	10	100101
Aug 17, 2026 @ [redacted]	Kali-Lab	T1548.003	Privilege Escalation, Defense Evasion	CUSTOM: Multiple Successful Sudo to ROOT Executions	10	100101

Figure 41 - A longer run of the same rule, showing how frequently it triggers under sustained activity.

8.3 Rule 100103 - new local account created

Property	Value
Rule ID	100103
Severity level	12
Logic	Fires on a single matching event
Parent event	5902 - new user added to the system, matched on useradd
ATT&CK technique	T1136.001 - Create Account: Local Account (Persistence)

What it detects: creation of a new local user account on the monitored host.

Why it matters: this is a persistence move. An attacker who has escalated once will often create their own account so that they keep access even if the original compromised credential is reset. Unlike the two rules above there is no frequency condition, because one unexplained account is already worth investigating. In a production environment this rule would need an exception list for provisioning tools, otherwise every legitimate onboarding would raise a level 12 alert.

timestamp	agent.name	rule.description	rule.level	rule.id
Aug 17, 2026 @ [redacted]	Kali-Lab	CUSTOM: New User Account Creation Detected	12	100103

Figure 42 - Rule 100103 firing at level 12 on account creation.

8.4 Rule 100102 - system discovery commands

Property	Value
Rule ID	100102
Severity level	8

Property	Value
Logic	Fires on a single event matching whoami, uname or hostname
ATT&CK technique	T1033 - System Owner/User Discovery

What it detects: basic reconnaissance commands run on the endpoint.

Why it matters: these are the first commands most attackers run after landing on a host, and they are cheap to detect. I wrote this one first because it has no frequency condition and therefore fires immediately, which made it the fastest way to prove my rule file was being loaded at all before I spent time on the harder rules. I have kept it in the file but I would not present it as one of my three headline detections: on a machine where an administrator works interactively it produces far too much noise to be useful without tuning, and command auditing has to be enabled for it to see anything at all.

9. Alert volume dashboard

The optional part of the task asked for a dashboard summarising alert volume by type. I built one from four panels: a time series of custom alerts, a total count, a breakdown by rule ID and a distribution donut. I filtered it to my three main rule IDs so the custom content is not buried under the built-in ruleset.

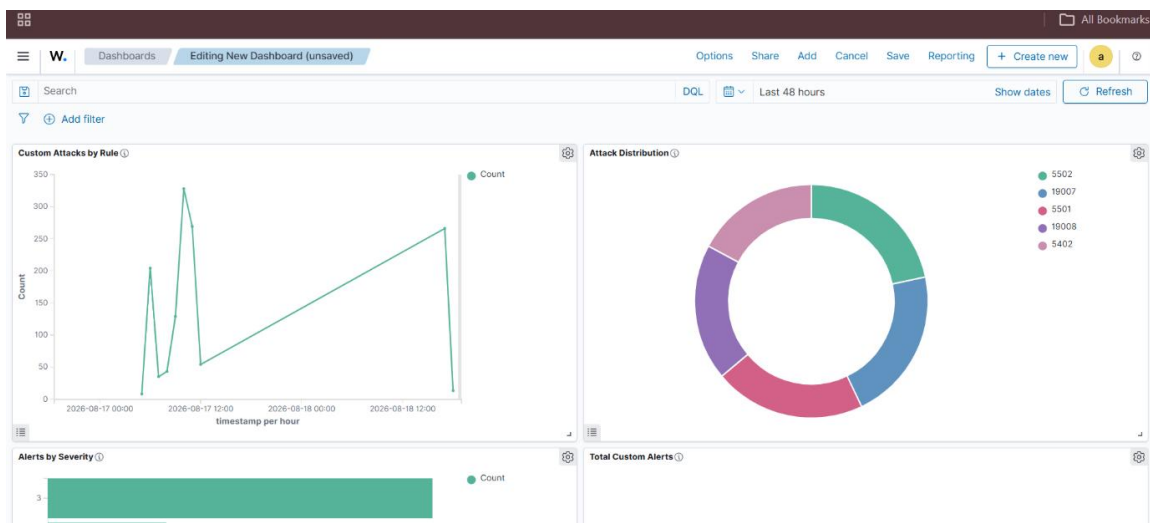


Figure 43 - Starting the dashboard against the unfiltered alert index, where built-in rules dominate the counts.

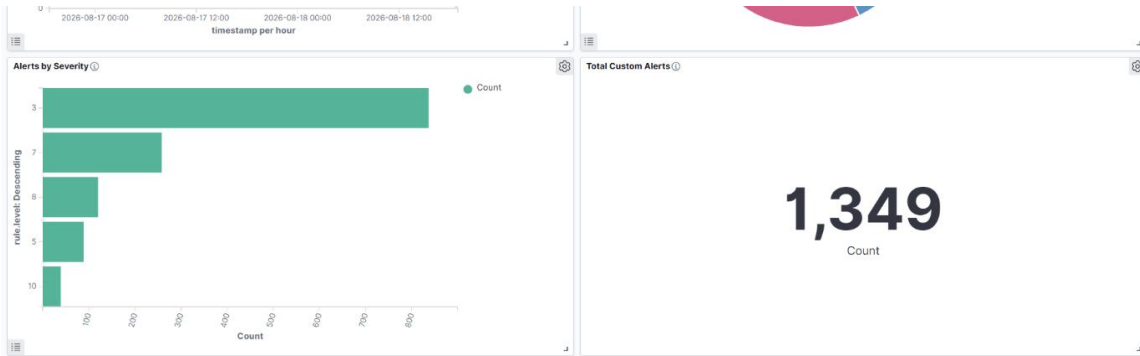


Figure 44 - The same panels before filtering, with over 1,300 alerts across all rules.

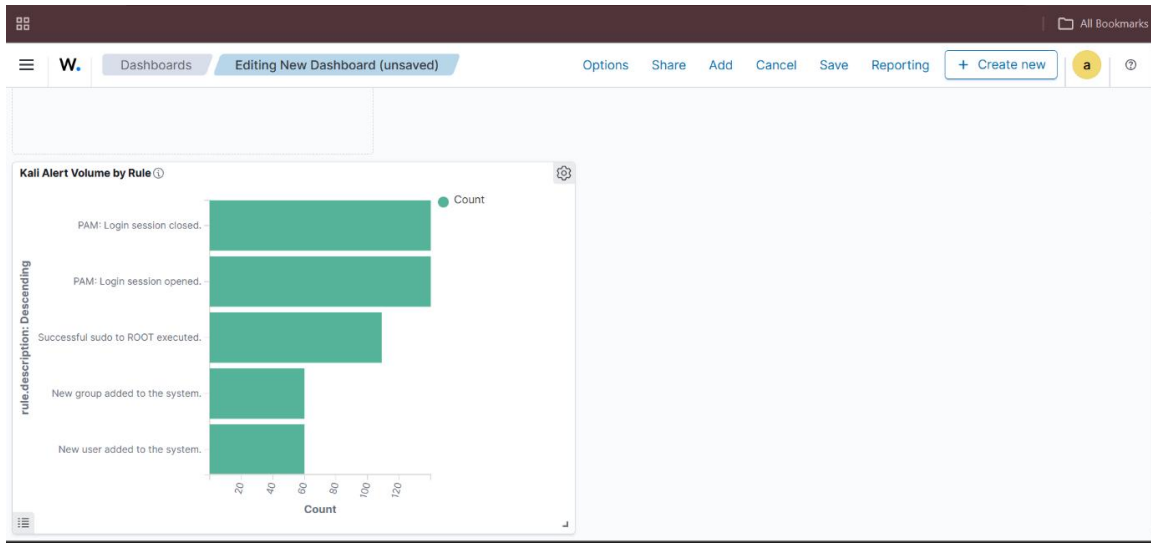


Figure 45 - A panel broken out by rule description, showing which event types are driving the volume.

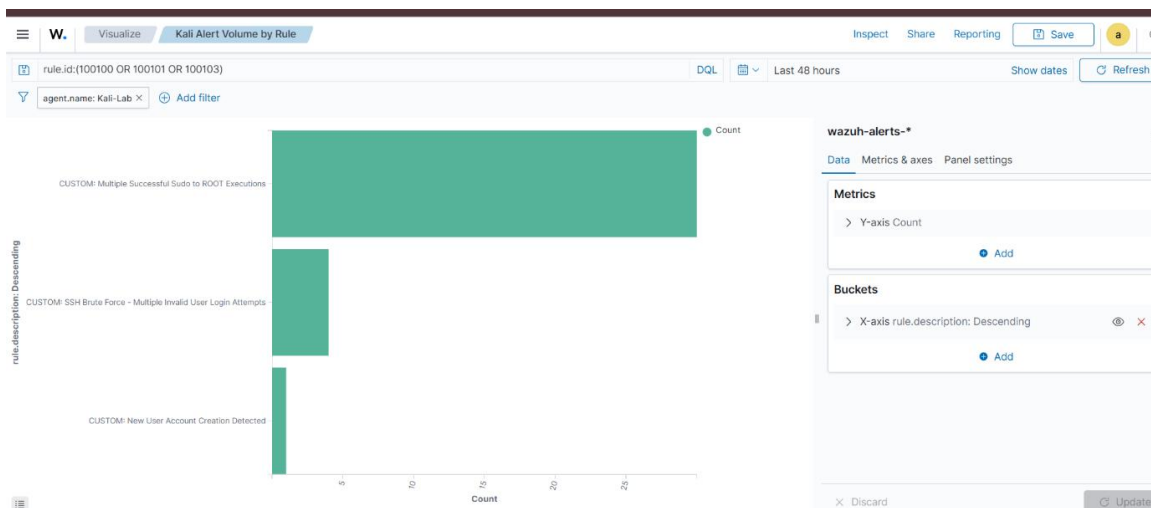


Figure 46 - Alert volume by rule, filtered to the agent and to my custom rules.

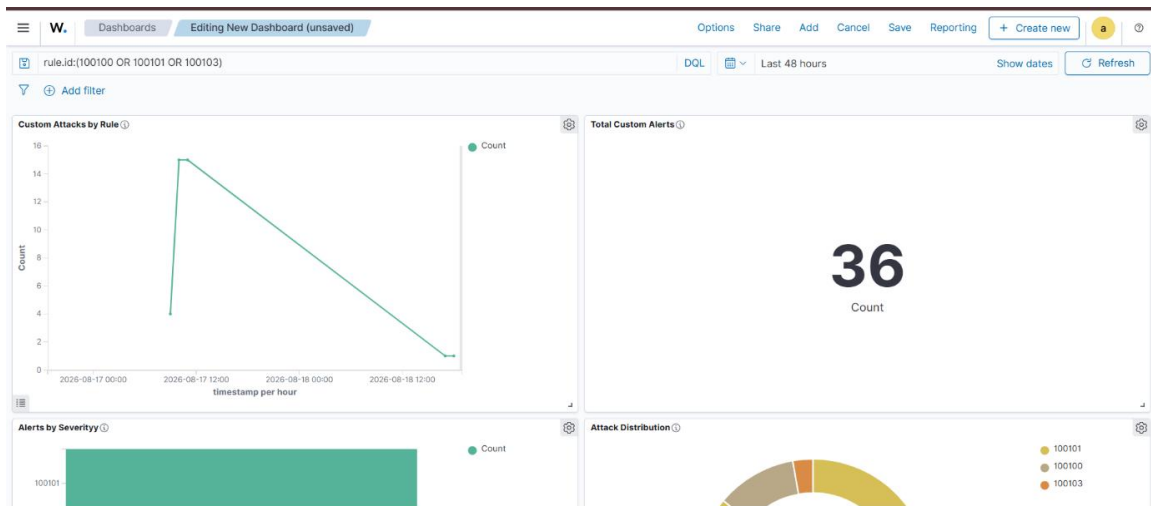


Figure 47 - The dashboard after filtering to rule IDs 100100, 100101 and 100103. Thirty-six custom alerts over 48 hours.

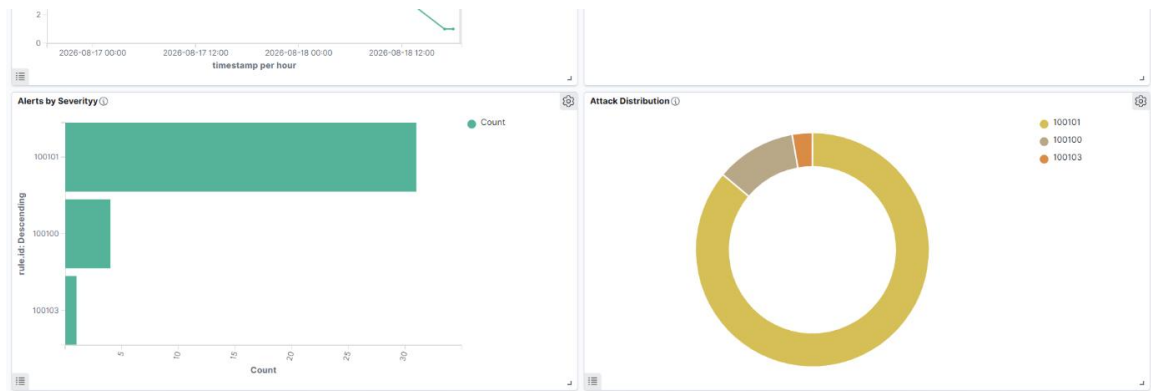


Figure 48 - Severity and attack distribution panels. Rule 100101 accounts for most of the volume, which matches its lower severity and looser threshold.

W. Dashboards

+ Create Dashboard

Search...

Title	Type	Description	Last updated	Actions
SOC-DASHBOARD-CUSTOM RULE	Dashboard		Aug 18, 2026 @ [redacted]	[edit icon]

Rows per page: 20

Figure 49 - The saved dashboard.

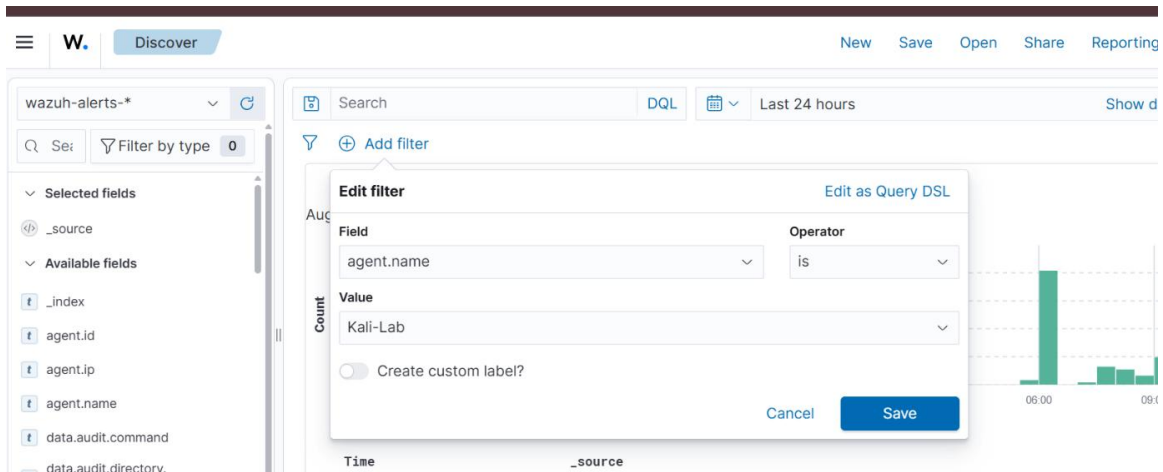


Figure 50 - The agent filter applied to the alert index, used across the dashboard panels.

10. MITRE ATT&CK mapping

Each rule carries its ATT&CK technique inside the rule definition itself, in the mitre block, rather than being mapped in a spreadsheet afterwards. That means the technique travels with the alert and can be filtered on in the dashboard, which is what makes the built-in ATT&CK view populate.

Rule	Detects	Technique	Tactic	Analyst action
100100	SSH brute force	T1110	Credential Access	Investigate the source address
100101	Repeated sudo to root	T1548.003	Privilege Escalation	Review sudo activity for the account
100102	System discovery	T1033	Discovery	Correlate with other activity on the host
100103	New local account	T1136.001	Persistence	Verify the account creation was authorised

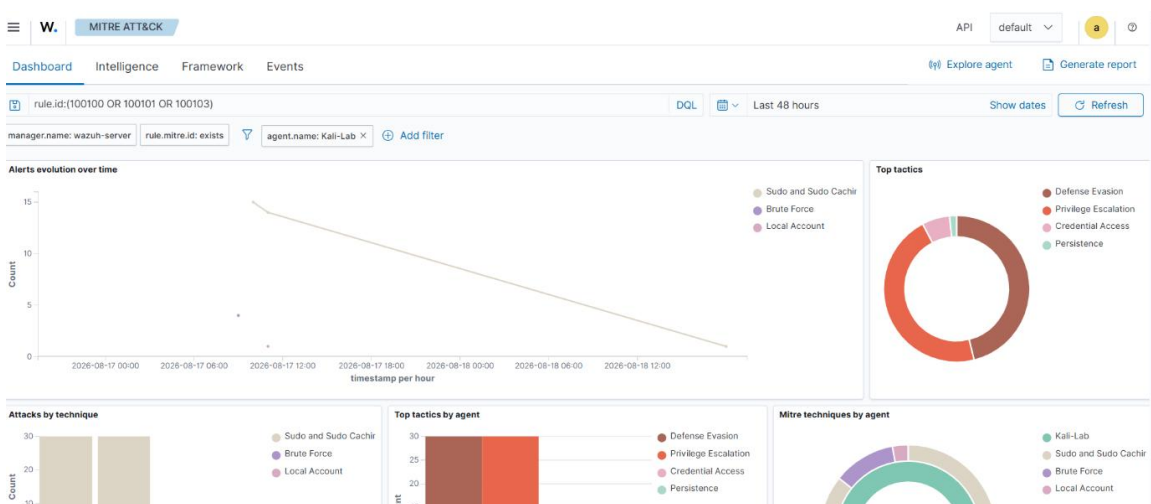


Figure 51 - The ATT&CK dashboard filtered to my custom rules.

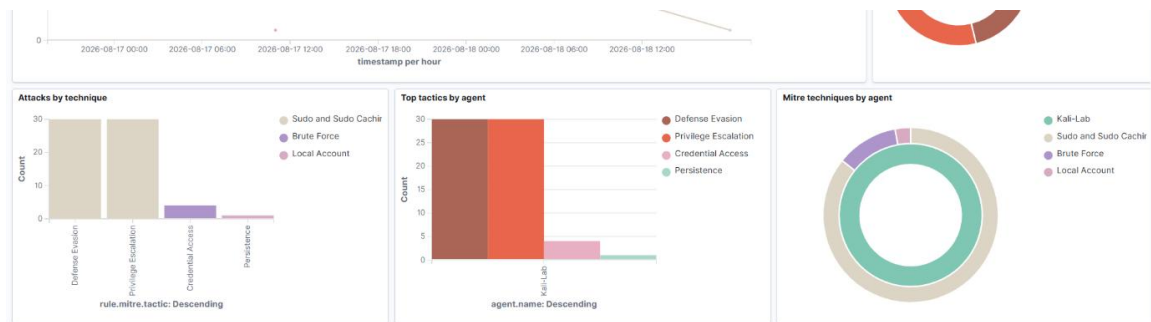


Figure 52 - Techniques and tactics broken down by agent.

Aug 16, 2026 @ [redacted] - Aug 18, 2026 @ [redacted]						
Export Formatted	Reset view	642 available fields	Columns	Density	1 fields sorted	Full screen
timestamp	agent.name	rule.mitre.id	rule.mitre.tactic	rule.description	rule.level	rule.id
Aug 18, 2026 @ [redacted]	Kali-Lab	T1548.003	Privilege Escalation, Defense Evasion	CUSTOM: Multiple Successful Sudo to ROOT Executions	10	100101
Aug 17, 2026 @ 11:37:43.0...	Kali-Lab	T1136.001	Persistence	CUSTOM: New User Account Creation Detected	12	100103
Aug 17, 2026 @ [redacted]	Kali-Lab	T1548.003	Privilege Escalation, Defense Evasion	CUSTOM: Multiple Successful Sudo to ROOT Executions	10	100101
Aug 17, 2026 @ [redacted]	Kali-Lab	T1548.003	Privilege Escalation, Defense Evasion	CUSTOM: Multiple Successful Sudo to ROOT Executions	10	100101
Aug 17, 2026 @ 11:18:33.8...	Kali-Lab	T1548.003	Privilege Escalation, Defense Evasion	CUSTOM: Multiple Successful Sudo to ROOT Executions	10	100101
Aug 17, 2026 @ [redacted]	Kali-Lab	T1548.003	Privilege Escalation, Defense Evasion	CUSTOM: Multiple Successful Sudo to ROOT Executions	10	100101
Aug 17, 2026 @ [redacted]	Kali-Lab	T1548.003	Privilege Escalation, Defense Evasion	CUSTOM: Multiple Successful Sudo to ROOT Executions	10	100101
Aug 17, 2026 @ [redacted]	Kali-Lab	T1548.003	Privilege Escalation, Defense Evasion	CUSTOM: Multiple Successful Sudo to ROOT Executions	10	100101
Aug 17, 2026 @ 11:07:02.6...	Kali-Lab	T1548.003	Privilege Escalation, Defense Evasion	CUSTOM: Multiple Successful Sudo to ROOT Executions	10	100101
Aug 17, 2026 @ [redacted]	Kali-Lab	T1548.003	Privilege Escalation, Defense Evasion	CUSTOM: Multiple Successful Sudo to ROOT Executions	10	100101
Aug 17, 2026 @ [redacted]	Kali-Lab	T1548.003	Privilege Escalation, Defense Evasion	CUSTOM: Multiple Successful Sudo to ROOT Executions	10	100101
Aug 17, 2026 @ [redacted]	Kali-Lab	T1548.003	Privilege Escalation, Defense Evasion	CUSTOM: Multiple Successful Sudo to ROOT Executions	10	100101
Aug 17, 2026 @ [redacted]	Kali-Lab	T1548.003	Privilege Escalation, Defense Evasion	CUSTOM: Multiple Successful Sudo to ROOT Executions	10	100101
Aug 17, 2026 @ 11:00:58.8...	Kali-Lab	T1548.003	Privilege Escalation, Defense Evasion	CUSTOM: Multiple Successful Sudo to ROOT Executions	10	100101
Aug 17, 2026 @ [redacted]	Kali-Lab	T1548.003	Privilege Escalation, Defense Evasion	CUSTOM: Multiple Successful Sudo to ROOT Executions	10	100101

Figure 53 - Alert table with the technique and tactic resolved against each custom alert.

MITRE ATT&CK Coverage

Rule	Attack	MITRE	Tactic	SOC Action
100100	SSH Brute Force	T1110	Credential Access	Investigate source IP
100101	Sudo/Root Abuse	T1548.003	Privilege Escalation	Review sudo activity
100103	New Local Account	T1136.001	Persistence	Verify account creation

Figure 54 - Summary of coverage and the suggested first action for each rule.

11. Reproducing this build

Short version, for anyone repeating the lab:

- Import the Wazuh OVA into VirtualBox and attach it to a dedicated NAT network.
- Attach the endpoint VM to the same network and forward the dashboard port to the host.
- Confirm all three Wazuh services are running before opening the dashboard.
- Prove ICMP and the two agent ports work between the machines before installing anything.
- Install the agent, set the manager address and agent name in the agent config, then start the service.
- Confirm the agent shows as active on the manager before writing any rules.

- Enable SSH on the endpoint and fail a login on purpose to confirm events are arriving.
- Write the rules into `custom_rules.xml`, deploy them through the API, and read the file back to confirm.
- Generate the matching behaviour for each rule and screenshot the alert.

12. Limitations and what I would do next

This is a lab, and a few things would need to change before any of it went near a client environment. The frequency thresholds were tuned against traffic I generated myself, so they reflect my typing speed rather than a real baseline; on a production host they would need revisiting against a week of normal activity. Rule 100102 needs command auditing enabled and would be too noisy to keep as written. Rule 100103 needs an allow-list for provisioning accounts. The lab also has a single agent, so I have not tested how the rules behave when the same technique appears across several hosts at once, which is where correlation usually gets interesting.

If I carried this further, the next steps would be to add a Windows endpoint so the same techniques can be detected on a second platform, to add active response so that a repeated brute force blocks the source automatically, and to version the rule file in Git so that changes to detection content are re-viewable.